Data Engineering Cheat Sheet – ShopKart edition (Module 7)

Tool: Google Colab. Upload `orders_raw.csv`, `customers.csv`, `products.csv`, `new_orders_2026_01.csv`. Full code: `Module_07_Data_Engineering/project/shopkart_pipeline.py` (run with `!python shopkart_pipeline.py`). No installs: `sqlite3` comes with Python.

Words to know

Term	Meaning	ShopKart example
Pipeline	Steps that move data from where it is created to where it is used, automatically	Raw CSVs → `shopkart_warehouse.db`
Extract	Read the source data as it is; change nothing	`pd.read_csv("orders_raw.csv")` → 5,055 rows
Transform	Clean, fix types, merge, add columns	The Module 6 cleaning → 5,000 rows, 21 columns
Load	Write the result where people query it	`to_sql(...)` into the warehouse
Data quality check	A test that **stops** the pipeline if the data looks wrong	No duplicate order_id, unit_price > 0
Batch	Move a chunk of data on a schedule	All of yesterday's orders, every night at 2 am
Streaming	Move each event within seconds	Live delivery tracking
Incremental load	Add only rows the warehouse has not seen	January 2026: add 36, skip 2 already loaded

ETL vs ELT

	ETL	ELT
Order	Extract → Transform → Load	Extract → Load → Transform
Where the cleaning runs	In Python (or an ETL tool) before loading	Inside the warehouse, usually as SQL (often with dbt)
Common with	Smaller data, on-premise databases	Cloud warehouses: BigQuery, Snowflake, Redshift
In this course	Module 7 pipeline	Named in 7.3 only

Both are normal. Same steps, different order.

Pipeline skeleton (pandas + sqlite3)

```
import sqlite3
import pandas as pd
```

```
def extract():
    return pd.read_csv("orders_raw.csv"),
pd.read_csv("customers.csv"), pd.read_csv("products.csv")

def transform(orders, customers, products):
    df = orders.drop_duplicates()
    # ... the Module 6 cleaning steps (city, ₹ prices, dates, payment,
discount, returned, qty 99) ...
    df = df.merge(products[["product_id", "product_name", "category",
"cost_price"]], on="product_id", how="left")
    df = df.merge(customers[["customer_id", "gender", "age",
"membership"]], on="customer_id", how="left")
    df["revenue"] = (df["quantity"] * df["unit_price"] * (1 -
df["discount"])).round(2)
    return df

def quality_checks(df, expected_rows):
    checks = {
        "row count = unique orders in source": len(df) ==
expected_rows,
        "no duplicate order_id": df["order_id"].is_unique,
        "every product found": df["category"].notnull().all(),
        "unit_price > 0": (df["unit_price"] > 0).all(),
    }
    failed = [name for name, ok in checks.items() if not ok]
    if failed:                                    # raise, don't just
print: nothing gets loaded
        raise ValueError("Data quality check failed: " + ";
".join(failed))

def load(fact_orders):
    conn = sqlite3.connect("shopkart_warehouse.db")
    fact_orders.to_sql("fact_orders", conn, if_exists="replace",
index=False)   # full rebuild
    conn.close()
```

Run order: `orders, customers, products = extract()` → `clean = transform(...)` → `quality_checks(clean, orders["order_id"].nunique())` → `load(...)`. The real pipeline has 7 checks (also: no missing keys, every customer found, quantity 1–10) and loads four tables.

Incremental load (safe to re-run)

```
conn = sqlite3.connect("shopkart_warehouse.db")
loaded_ids = set(pd.read_sql("SELECT order_id FROM fact_orders", conn)
["order_id"])
to_add = new_df[~new_df["order_id"].isin(loaded_ids)]       # ~ means
NOT: keep unseen order_ids
to_add.to_sql("fact_orders", conn, if_exists="append", index=False)
conn.close()
```

if_exists=	What happens	Use for
`"replace"`	Drops the table and writes only these rows	Full rebuild (`load()`)
`"append"`	Adds these rows to the table	Incremental load
`"fail"` (default)	Error if the table exists	Safety

ShopKart: first run adds 36 and skips 2 (fact_orders 5,000 → 5,036); a second run adds 0 and skips 38.

Star schema

One **fact** table (events and numbers) in the middle, **dimension** tables (descriptions) around it, joined by keys.

Table	Rows	Holds
`fact_orders`	5,000 → 5,036	order_id, date_key, customer_id, product_id, quantity, unit_price, discount, city, payment_mode, delivery_days, rating, returned, revenue, profit
`dim_customer`	800	name, gender, age, city, signup_date, membership
`dim_product`	60	product_name, category, list_price, cost_price
`dim_date`	336 → 357	date_key (20251031), date, year, month, month_name, quarter, weekday, is_weekend

```sql
SELECT d.year, d.month, p.category, COUNT(*) AS orders,
ROUND(SUM(f.revenue), 2) AS revenue
FROM fact_orders AS f
JOIN dim_product AS p ON f.product_id = p.product_id
JOIN dim_date    AS d ON f.date_key   = d.date_key
GROUP BY d.year, d.month, p.category
ORDER BY d.year, d.month, revenue DESC;
```

Check: 2025 = 5,000 orders, ₹1,47,04,649.70 (same as Modules 2, 4 and 6); January 2026 = 36 orders, ₹1,73,149.15.

OLTP vs OLAP

	OLTP database	OLAP warehouse
Job	Runs the app: place an order, update stock	Answers analysis questions over lots of history
Writes	Many small writes, all day	Big loads on a schedule (e.g. nightly)
Typical query	One customer's last order	Revenue by month and category for 2 years
ShopKart	The shop app's own database	`shopkart_warehouse.db`

Run heavy reports on the warehouse, not on the live app database.

Scheduling and the tool names

Tool	What it is for	In this course
cron	Run a command on a timetable: `0 2 * * *` = every day at 2:00 am	Shown in 7.3
Apache Airflow	Pipelines as DAGs (tasks + arrows) with retries and alerts	DAG shown on a slide, not run
Apache Spark	Process data too big for one machine, across many	Named only
BigQuery, Snowflake, Redshift	Cloud data warehouses	Named only
dbt	Transformations written as SQL inside the warehouse (the T in ELT)	Named only

Common mistakes

Mistake	Fix
Printing FAIL instead of raising an error	`raise ValueError(...)` so bad data is never loaded
`if_exists="replace"` in the incremental load	Use `"append"`; replace leaves only the new rows
Forgetting `~`	You would add the old orders and skip the new ones
`no such table: fact_orders` after a Colab reset	The `.db` file was deleted; run all cells from the top
Changed a function, nothing changed	Re-run the cell with the `def`, then the cells that use it